SAT编码和CDCL算法

Outline

- SAT Encoding
- SAT Solving Basis
- CDCL Algorithm

Hello, SAT!

- Propositional Satisfiability, aka. Boolean Satisfiability, abbreviated as SAT

- The first NP-Complete problem [Cook, 1971]
- A core problem in computer science and a basic problem in logic
- SAT solvers widely used in industry and science

> The SAT problem is evidently a killer app, because it is key to the solution of so many other problems. SAT-solving techniques are among computer science's best success stories so far, and these volumes tell that fascinating tale in the words of the leading SAT experts.
>
> *Donald Knuth*

> . . . Clearly, efficient SAT solving is a key technology for 21st century computer science. I expect this collection of papers on all theoretical and practical aspects of SAT solving will be extremely useful to both students and researchers and will lead to many further advances in the field.
>
> *Edmund Clarke*

SAT

Propositional Satisfiability (SAT): Given a propositional formula φ, test whether there is an assignment to the variables that makes φ true.

e.g., $\varphi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (x_2 \vee \neg x_4) \wedge (\neg x_1 \vee \neg x_3 \vee x_4)$

- Boolean variables: $x_1, x_2, \ldots$
- A literal is a Boolean variable x (positive literal) or its negation $\neg x$ (negative literal)
- A clause is a disjunction ($\vee$) of literals

  $x_2 \vee x_3$,

  $\neg x_1 \vee \neg x_3 \vee x_4$ $(\{\neg x_1, \neg x_3, x_4\})$

- A Conjunctive Normal Form(CNF) formula is a conjunction ($\wedge$) of clauses.

e.g., $\varphi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (x_2 \vee \neg x_4) \wedge (\neg x_1 \vee \neg x_3 \vee x_4)$

Every propositional formulas can be converted into CNF efficiently.

SAT

Example: Determine the satisfiability of the following compound propositions:

$$(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p)$$

Solution: Satisfiable. Assign **T** to *p*, *q*, and *r*.

$$(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p) \wedge (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$$

Solution: Not satisfiable. Check each possible assignment of truth values to the propositional variables and none will make the proposition true.

SAT solvers

- Using SAT solvers
  - To find a certain structure
  - To prove something

Problem Solving
Scheduling,
Resource allocation,
Logistics,
Hardware design,
Software Engineering,
…

Automatic Reasoning
Theorem proving,
System verification,
Knowledge representation,
Decision procedure,
Agents,
…

Finding a solution

Prove that
$x \geq 1, y \geq 1 \Rightarrow x + y \geq 2$

Is
$x \geq 1, y \geq 1, not(x + y \geq 2)$
UNSAT?

Checking validity

SAT Revolution

EDA

original C code

```
if(!a && !b) h();
else if(!a) g();
else f();

        ⇓

if(!a) {
  if(!b) h();
  else g();
} else f();
```

optimized C code

```
if(a) f();
else if(b) g();
else h();

        ⇑

if(a) f();
else {
  if(!b) h();
  else g(); }
```

How to check that these two versions are equivalent?

Program Analysis

Planning

cryptography

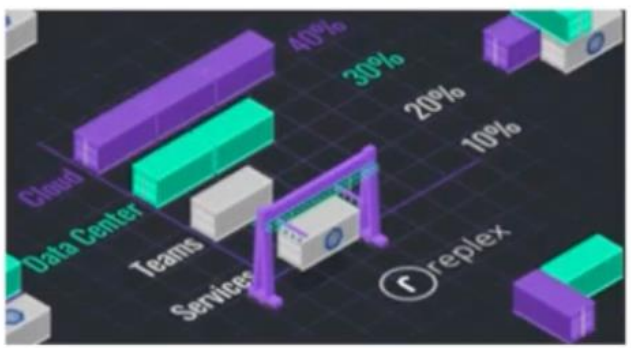
$x^m + y^m = z^m \pmod p$ $vdW(6) = 1132$

Schur's Theorem *Ramsey Theory*

Pythagorean Tuples Conjecture

3n+1 Conjecture?

Math



Resource Allocation

Using SAT Solvers

Input file: DIMACS format.

```
c example
p cnf 4 4
1 -4 -3 0
1 4 0
-1 0
-4 3 0
```

lines starting "c" are comments and are ignored by the SAT solver.
a line starting with "p cnf" is the problem definition line containing the number of variables and clauses.
the rest of the lines represent clauses, literals are integers (starting with variable 1), clauses are terminated by a zero.

Using SAT Solvers

Input file: DIMACS format.

```
c example
p cnf 4 4
1 -4 -3 0
1 4 0
-1 0
-4 3 0
```

lines starting "c" are comments and are ignored by the SAT solver.
a line starting with "p cnf" is the problem definition line containing the number of variables and clauses.
the rest of the lines represent clauses, literals are integers (starting with variable 1), clauses are terminated by a zero.

Output format

```
c comments, usually stastitics about the solving
s SATISFIABLE
v 1 2 -3 -4  5 -6 -7 8 9 10
v -11 12 13 -14 15 0
```

the solution line (starting with "s") can contain SATISFIABLE, UNSATISFIABLE and UNKNOWN.
For SATISFIABLE case, the truth values of variables are printed in lines starting with "v", the last value is followed by a "0"

UNSAT certificate will be introduced later.

SAT Encoding: Logic Puzzle

- Question: at least one of them speak truth. Who speaks the truth?
  - A: B is lying.
  - B: C is lying.
  - C: A and B are lying.

SAT Encoding: Logic Puzzle

- Question: at least one of them speak truth. Who speaks the truth?
  - A: B is lying.
  - B: C is lying.
  - C: A and B are lying.

- Encoding:
  - 3 variables: a, b, c present A, B, C speak truth, while ¬a, ¬b, ¬c present lying.

SAT Encoding: Logic Puzzle

- Question: at least one of them speak truth. Who speaks the truth?
  - A: B is lying.
  - B: C is lying.
  - C: A and B are lying.

- Encoding:
  - 3 variables: a, b, c present A, B, C speak truth, while ¬a, ¬b, ¬c present lying.
  - clauses:

a ∨ b ∨ c ; %at least one speak truth.

¬ a ∨ ¬ b; a ∨ b; %a-> ¬b, ¬a -> b

¬ b ∨ ¬ c; b ∨ c; %b-> ¬c, ¬b -> c

¬ c ∨ ¬ a; ¬c ∨ ¬b; c ∨ a ∨ b %c->(¬a∧ ¬b), ¬c->¬ (¬a∧ ¬b)

SAT Encoding: Logic Puzzle

- Question: at least one of them speak truth. Who speaks the truth?
  - A: B is lying.
  - B: C is lying.
  - C: A and B are lying.

- Encoding:
  - 3 variables: a, b, c present A, B, C speak truth, while ¬a, ¬b, ¬c present lying.
  - clauses:

a ∨ b ∨ c ; %at least one speak truth.
¬ a ∨ ¬ b; a ∨ b; %a-> ¬b, ¬a -> b
¬ b ∨ ¬ c; b ∨ c; %b-> ¬c, ¬b -> c
¬ c ∨ ¬ a; ¬c ∨ ¬b; c ∨ a ∨ b %c->(¬a∧ ¬b), ¬c->¬ (¬a∧ ¬b)

- Result: ¬a, b, ¬c

  B speaks truth, A and C are lying

SAT Encoding: Sudoku

- A **Sudoku puzzle** is represented by a 9×9 grid made up of nine 3×3 blocks. Some of the 81 cells of the puzzle are assigned one of the numbers 1,2, …, 9.
- Goal: assign numbers to each blank cell so that every row, column and block contains each of the nine possible numbers.

- Let $p(i,j,n)$ denote the proposition that is true when the cell in the i-th row and the j-th column has number n.
- There are $9 \times 9 \times 9 = 729$ such propositions.
- In the sample puzzle p(5,1,6) is true, but p(5,j,6) is false for j = 2,3,…9

	2	9				4		
			5			1		
	4							
				4	2			
6							7	
5								
7			3					5
	1			9				
							6	

SAT Encoding: Sudoku

- For each cell with a given value, assert $p(i,j,n)$, when the cell in row i and column j has the given value.

- Assert that every row contains every number.

$$\bigwedge_{i=1}^{9} \bigwedge_{n=1}^{9} \bigvee_{j=1}^{9} p(i,j,n)$$

- Assert that every column contains every number.

$$\bigwedge_{j=1}^{9} \bigwedge_{n=1}^{9} \bigvee_{i=1}^{9} p(i,j,n)$$

	2	9				4		
			5			1		
	4							
				4	2			
6							7	
5								
7			3					5
	1			9				
							6	

SAT Encoding: Sudoku

- Assert that each of the 3 x 3 blocks contain every number.

$$\bigwedge_{r=0}^{2}\bigwedge_{s=0}^{2}\bigwedge_{n=1}^{9}\bigvee_{i=1}^{3}\bigvee_{j=1}^{3} p(3r+i,3s+j,n)$$

- Assert that no cell contains more than one number.

$$n \neq n'$$

$$\neg p(i,j,n) \vee \neg p(i,j,n')$$

通过枚举二元负文字对保证独一性

	2	9				4		
			5			1		
	4							
				4	2			
6							7	
5								
7			3					5
	1			9				
							6	

- SAT Encoding
- SAT Solving Basis
- CDCL Algorithm

Tractable and intractable classes

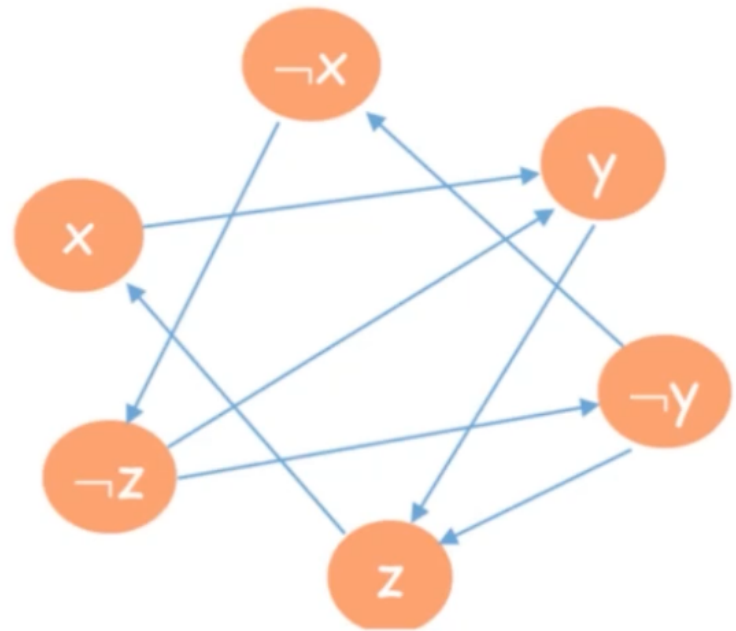
- SAT is generally NP complete
  - some classes of the problem can be solved within polynomial time (2SAT, Horn-SAT).
  - Some classes are NP complete (3-SAT)

$\Psi = (\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z) \wedge (z \vee y)$

- Solving 2-SAT problem is easy

- For each clause $l \vee k$, produce two edges $\neg l \rightarrow k, \neg k \rightarrow l$

Check whether there a variable x, such that there is loop between x *and* $\neg x$.



Dichotomy theorem of SAT

- SAT is generally NP complete
  - some classes of the problem can be solved within polynomial time (2SAT, Horn-SAT).
  - Some classes are NP complete (3-SAT)
- Are there any criteria to distinguish between tractable and intractable classes?

Theorem (Schaefer, STOC 1978)

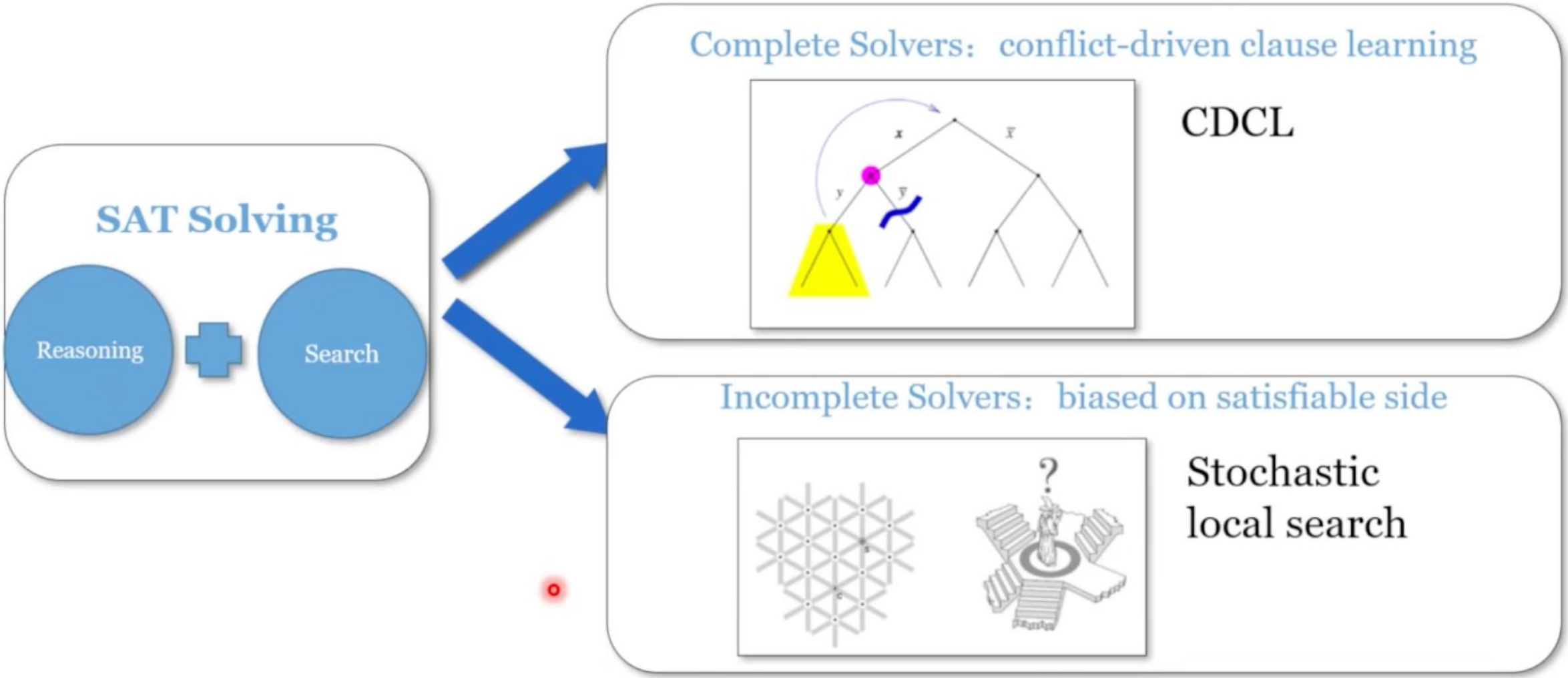
Given a Boolean constraint set C, SAT(C) is in P if C satisfies one condition below, and otherwise it is NP-complete:

- 1. C is 0-valid (1-valid);
- 2. C is weakly positive (weakly negative); // Horn, dual-Horn
- 3. C is bijunctive; //2-SAT
- 4. C is affine. //A system of linear equations

A CNF formula is Horn (dul-Horn, resp.) if every clause in this formula has at most one positive (negative, resp.) literal

对 m 个子句、n 个变量，算法最多执行 O(m+n) 步。

SAT Solving Basis



SAT Solving Basis – Resolution

Resolution

$$\frac{C \vee x \qquad \bar{x} \vee D}{C \vee D}\ [\text{Res}]$$

SAT Solving Basis – Resolution

Algorithm 1: Saturation Algorithm

Input: CNF formula F
Output: SAT,UNSAT

```
1  while true do
2      R = resolveAll(F)
3      if R ∩ F ≠ R then
4          F = F ∪ R
5      else
6          break
7  if ⊥ ∈ F then
8      return UNSAT
9  else
10     return SAT
```

基于归结（resolution）的SAT验证

对F中所有可以归结的子句进行归结，得到的子句记为R
讲R中新的子句加入到F中，
当R已经没有新的子句（也就是R包含于F）的时候，认为子句集已经饱和了，此时break

终止判断：看F是否有空子句（就是那个奇怪的符号）
如果有，意味着推出了矛盾，则为UNSAT
如果没有，那就认为是SAT

- A deductive system based on the binary-resolution rule as its single inference rule is sound and complete.
- In other words, a CNF formula is unsatisfiable if and only if there exists a finite series of binary-resolution steps ending with the empty clause.

- Has exponential time and space complexity (always for Pigeons)

SAT Solving Basis - Resolution

Variable elimination by resolution

- Given a formula F and a literal a, the formula denoted $DP_a(F)$ is constructed from F
  - by adding all resolvents by a
  - and then removing all clauses that contain a or $\neg a$

Example. $F = (x \vee e) \wedge (y \vee e) \wedge (\bar{x} \vee z \vee \bar{e}) \wedge (y \vee \bar{e}) \wedge (y \vee z)$

Eliminating variable e by resolution:

- first add all resolvents upon e.

$\{(x \vee e), (y \vee e)\}$ with $\{(\bar{x} \vee z \vee \bar{e}), (y \vee \bar{e})\} \rightarrow$ 4 resolvents

$$F \wedge (x \vee \bar{x} \vee z) \wedge (x \vee y) \wedge (y \vee \bar{x} \vee z) \wedge (y)$$

- remove all clauses that contain e to obtain

$$(y \vee z) \wedge \cancel{(x \vee \bar{x} \vee z)} \wedge (x \vee y)\,(y \vee \bar{x} \vee z) \wedge (y)$$

SAT Solving Basis - Resolution

- $F = (x \vee e) \wedge (y \vee e) \wedge (\bar{x} \vee z \vee \bar{e}) \wedge (y \vee \bar{e}) \wedge (y \vee z)$

  $\equiv ((x \wedge y) \vee e) \wedge (((\bar{x} \vee z) \wedge y) \vee \bar{e}) \wedge (y \vee z)$

$((x \wedge y) \vee e) \wedge (((\bar{x} \vee z) \wedge y) \vee \bar{e})$

$\equiv (x \wedge y) \vee ((\bar{x} \vee z) \wedge y)$

$\equiv [(x \wedge y) \vee (\bar{x} \vee z)] \wedge [(x \wedge y) \vee y]$

$\equiv [(\bar{x} \vee z \vee x) \wedge (\bar{x} \vee z \vee y)] \wedge [(x \vee y) \wedge (y \vee y)]$

$\equiv (\bar{x} \vee z \vee y) \wedge (x \vee y) \wedge y$

对于含有相同文字的子句：提取出文字e和-e
对于含有相反文字的子句，利用归结原理消解

SAT Solving Basis - Unit Propagation

- Unit Clause: A Clause that all literals are falsified except one unassigned literal.
- Unit Propagation (UP): the unassigned literal in unit clause can only be assigned to single value to satisfy the clause.

Example:

	x_1	T		x_1 sat
	x_2			$\bar{x}_1 \vee \bar{x}_2$
$Vars$:	x_3		$clauses$:	$\bar{x}_1 \vee \bar{x}_3$
	x_4			$x_2 \vee \bar{x}_3 \vee \bar{x}_4$
	x_5			$\bar{x}_1 \vee x_4 \vee x_5$
	x_6			$x_2 \vee x_4 \vee x_6$

Unit Propagation is also called Boolean Constraint Propagation

SAT Solving Basis – DP Algorithm

Davis-Putnam Algorithm [1960] % could find record in 1959

- Rule 1: Unit propagation
- Rule 2: Pure literal elimination
- Rule 3: Resolution at one variable

公式：

$F=(x1 \vee \neg x2) \wedge (x3 \vee x2) \wedge (x3 \vee x4)$

- x1：只出现为正文字 → pure literal ✅
- x4：只出现为正文字 → pure literal ✅
- x2：出现了正负两种形式 → not pure ❌
- x3：只出现为正文字 → pure literal ✅

Apply deduction rules (giving priority to rules 1 and 2) until no further rule is applicable

Unit Propagation（单子句传播）：如果单元传播，那么文字的赋值就被确定了

Pure Literal Elimination（纯文字消去）：直接赋值，那么所有含有这个文字的子句都可以被消去

pure literal：一个变量，只以同一种极性出现【就是右上方“只出现正文字”】

Resolution at one variable（单变量归结）：对含有x和-x的子句做归结

A brief history of SAT solving

- 1960-1990
  - DP, DPLL
  - NP completeness, Complexity, Tractable subclass,...
  - Resolution: Stålmarck's Method (1989)

A brief history of SAT solving

- 1960-1990
  - DP, DPLL
  - NP completeness, Complexity, Tractable subclass,...
  - Resolution: Stålmarck's Method (1989)
- 1990-2010
  - Local Search (1992): GSAT (1992), WalkSAT (1994)
  - Conflict Driven Clause Learning (1996): GRASP(1999), Chaff(2000), MiniSAT (2003), Glucose (2009)
  - Phase transition, Survey Propagation
  - Portfolio: SATzilla (2007)

A brief history of SAT solving

- 1960-1990
  - DP, DPLL
  - NP completeness, Complexity, Tractable subclass,...
  - Resolution: Stålmarck's Method (1989)
- 1990-2010
  - Local Search (1992): GSAT (1992), WalkSAT (1994)
  - Conflict Driven Clause Learning (1996): GRASP(1999), Chaff(2000), MiniSAT (2003), Glucose (2009)
  - Phase transition, Survey Propagation
  - Portfolio: SATzilla (2007)
- 2010~today
  - Modern local search: probSAT(2012), CCAnr(2013)
  - Advanced clause management and simplification
  - Solver engineering: Kissat(2019)
  - Hybridizing CDCL and local search (2020)
  - Parallel solving: cube and conquer

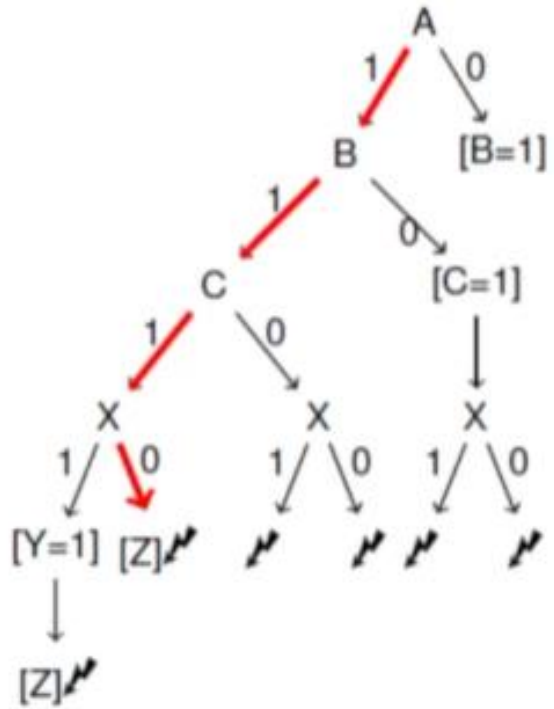
DPLL Algorithm

Davis-Putnam-Logemann-Loveland (DPLL, 1962)

- Chronological backtracking + UP + Decision heuristics

先假设一个值（如 x=T）看能不能走下去
不行就回溯（改猜x=F来尝试）

$\Delta =$
1. $\{A, B\}$
2. $\{B, C\}$
3. $\{\neg A, \neg X, Y\}$
4. $\{\neg A, X, Z\}$
5. $\{\neg A, \neg Y, Z\}$
6. $\{\neg A, X, \neg Z\}$
7. $\{\neg A, \neg Y, \neg Z\}$



Decision level

[UP] Decide [UP] Decide [UP] ....

Level 0 Level 1

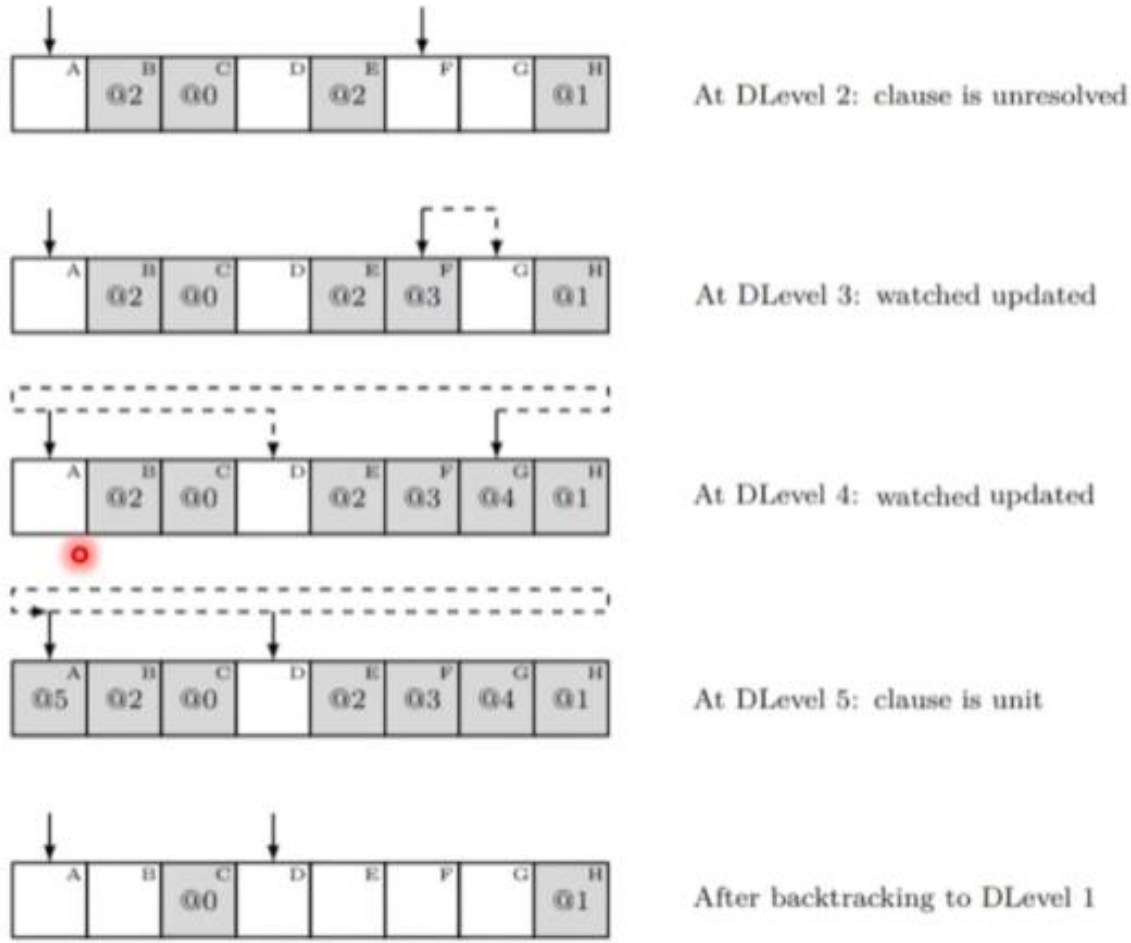
- SAT Encoding
- SAT Solving Basis
- CDCL Algorithm

Lazy data structure for UP

[ZhangStickel' 00] [MoskewiczMadiganZhaoZhangMalik' 01]

Efficient UP: 2 watched literals

- In each non-satisfied clause "watch" two non-false literals
- For each literal remember all the clauses where it is watched



VSIDS Branching heuristics

- Branching heuristics are used for deciding which variable to use when branching.
  - Main principle: Solvers prefer the variable which may cause conflicts faster.

- Variable State Independent Decaying Sum (VSIDS) [MoskewiczMadiganZhaoZhangMalik'01]
  - **Compute score for each variable, select variable with highest score**
  - Initial variable score is number of literal occurrences.
  - For a new conflict clause c: score of all variables in c is incremented.
  - Periodically, divide all scores by a constant. // forgetting previous effects

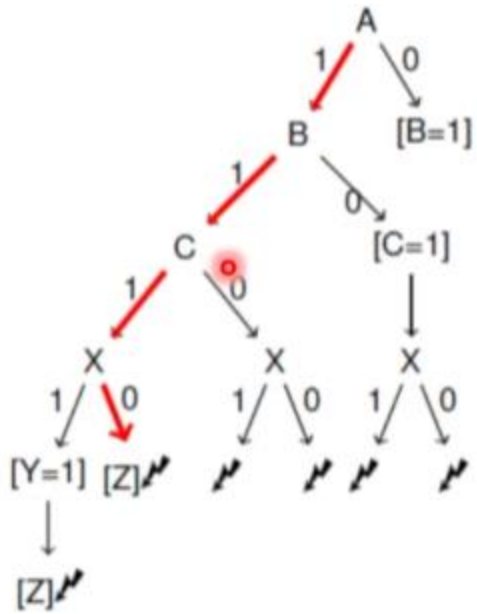
VSIDS Branching heuristics

- Most popular: the exponential variant in MiniSAT (EVSIDS)
  - The scores of some variables are *bumped* with *inc*, and *inc* decays after each conflict.
  - Initialize *score* to 0, the bump score *inc* default to 1.
  - *inc* multiply 1/*decay* after each conflict, *decay* initialized to 0.8, increased by 0.01 every [5k] conflicts, the maximum of *decay* is 0.95.
  - The score of variables in conflict clause *c* are bumped with *inc*.

Backjumping

$\Delta =$

1. $\{A, B\}$
2. $\{B, C\}$
3. $\{\neg A, \neg X, Y\}$
4. $\{\neg A, X, Z\}$
5. $\{\neg A, \neg Y, Z\}$
6. $\{\neg A, X, \neg Z\}$
7. $\{\neg A, \neg Y, \neg Z\}$

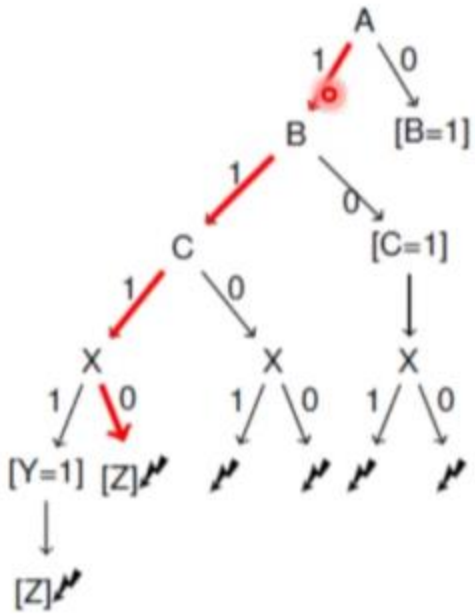


Chronological Backtracking

The first two conflicting clauses $\{\neg A, \neg X, Y\}, \{\neg A, X, \neg Z\}$ do not involve B and C.

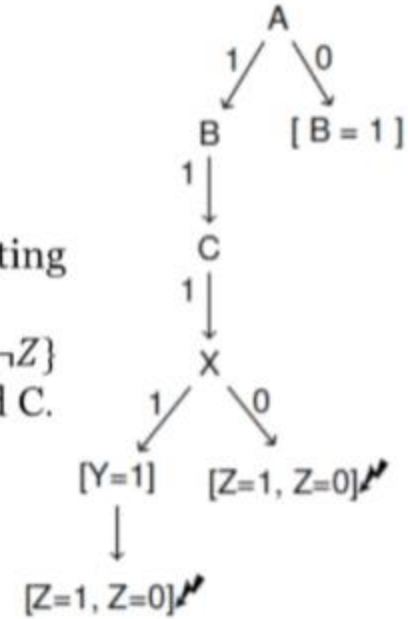
Backjumping

$\Delta =$
1. $\{A, B\}$
2. $\{B, C\}$
3. $\{\neg A, \neg X, Y\}$
4. $\{\neg A, X, Z\}$
5. $\{\neg A, \neg Y, Z\}$
6. $\{\neg A, X, \neg Z\}$
7. $\{\neg A, \neg Y, \neg Z\}$



Chronological Backtracking

The first two conflicting clauses $\{\neg A, \neg X, Y\}, \{\neg A, X, \neg Z\}$ do not involve B and C.



Non-Chronological Backtracking

CDCL: Conflict Driven Clause Learning

- A demonstration on clause learning

$$\varphi = (a \vee b) \wedge (\neg b \vee c \vee d) \wedge (\neg b \vee e) \wedge (\neg d \vee \neg e \vee f) \ldots$$

- Assume decisions $c = 0$ and $f = 0$
- Assign $a = 0$ and imply assignments
- A conflict is reached: $(\neg d \vee \neg e \vee f)$ is unsatisfied
- $(a = 0) \wedge (c = 0) \wedge (f = 0) \Rightarrow (\varphi = 0)$
- $(\varphi = 1) \Rightarrow (a = 1) \vee (c = 1) \vee (f = 1)$

- Learn new clause $(a \vee c \vee f)$

如果acf这样全赋值成0，那么就会导致冲突
所以学习子句为，如果这样赋值，那么就子句就为false

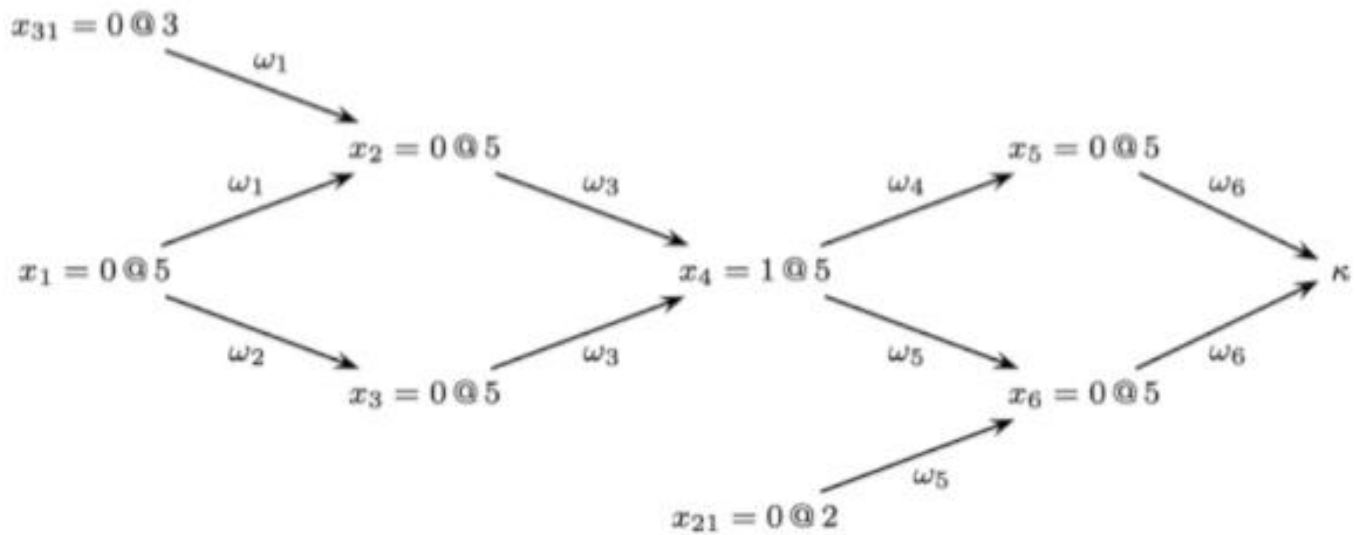
CDCL – Implication Graph

[MarqueSilvaSakallah' 96]

Implication Graph describes the decision and reasoning path.

- Vertex: (decision variable = value @decision level)
- Edge : unit clause used in UP (Reason Clause) .
- Conflict: all literals are falsified (under the current assignment).

$$\begin{aligned}\varphi_1 &= \omega_1 \wedge \omega_2 \wedge \omega_3 \wedge \omega_4 \wedge \omega_5 \wedge \omega_6 \\ &= (x_1 \vee x_{31} \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge \\ &\quad (\neg x_4 \vee \neg x_5) \wedge (x_{21} \vee \neg x_4 \vee \neg x_6) \wedge (x_5 \vee x_6)\end{aligned}$$



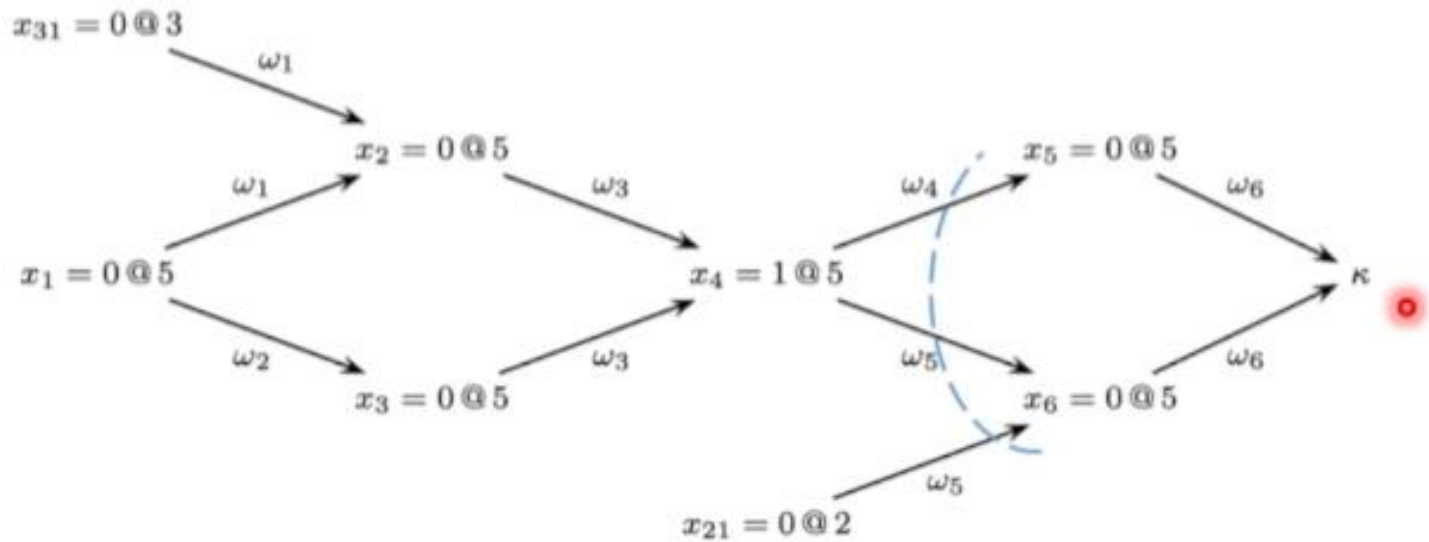
CDCL – Implication Graph

[MarqueSilvaSakallah' 96]

Implication Graph describes the decision and reasoning path.

- Vertex: (decision variable = value @decision level)
- Edge : unit clause used in UP (Reason Clause) .
- Conflict: all literals are falsified (under the current assignment).

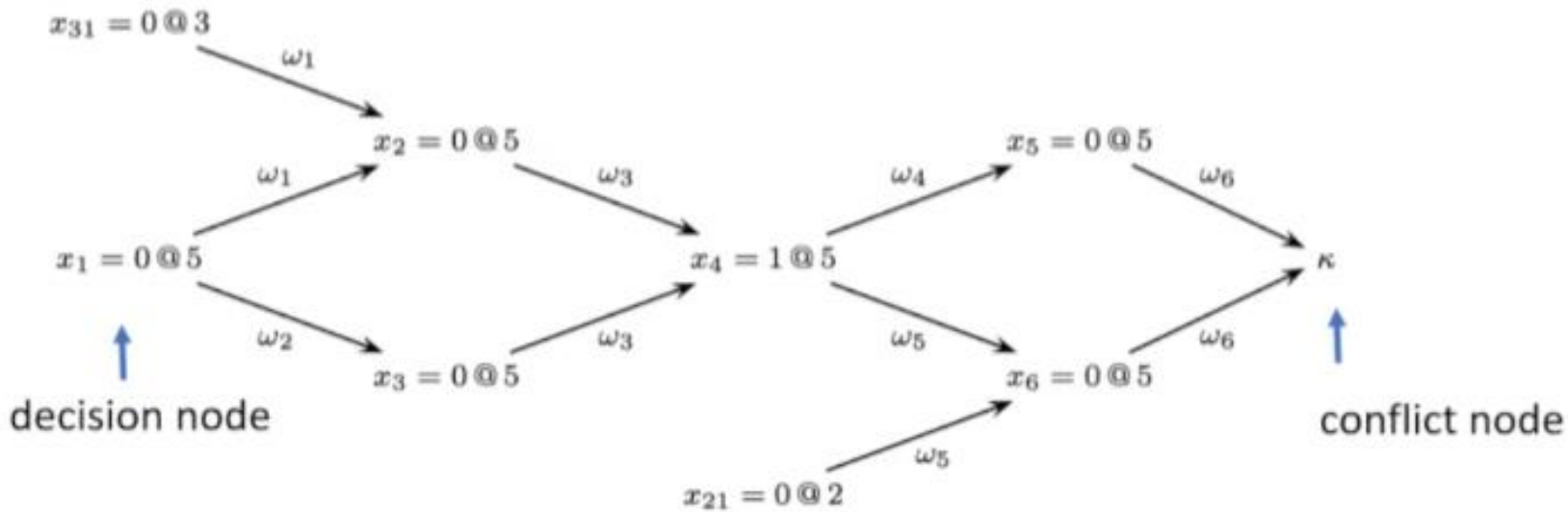
$$\begin{aligned}\varphi_1 &= \omega_1 \wedge \omega_2 \wedge \omega_3 \wedge \omega_4 \wedge \omega_5 \wedge \omega_6 \\ &= (x_1 \vee x_{31} \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge \\ &\quad (\neg x_4 \vee \neg x_5) \wedge (x_{21} \vee \neg x_4 \vee \neg x_6) \wedge (x_5 \vee x_6)\end{aligned}$$



Unique Implication Point

- A partial implication graph is a subgraph of an implication graph, which illustrates the BCP at a specific decision level.

- Given a partial conflict graph corresponding to the decision level of the conflict, a unique implication point (UIP) is any node other than the conflict node that is on all paths from the decision node to the conflict node.



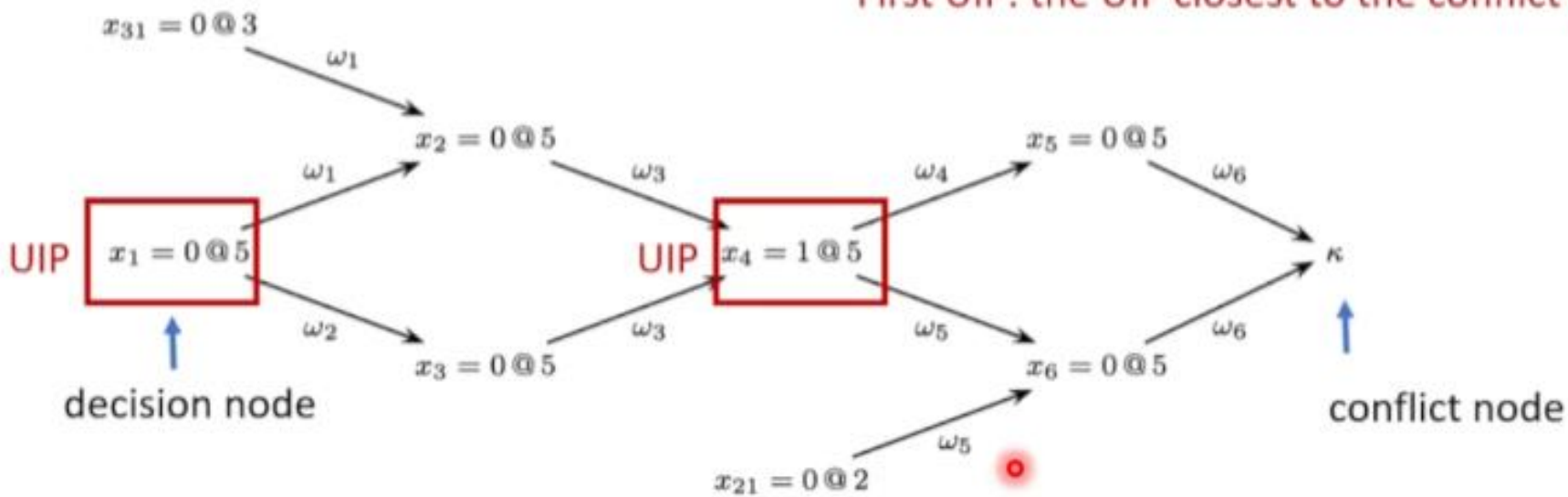
partial implication graph of level 5

Unique Implication Point

- A partial implication graph is a subgraph of an implication graph, which illustrates the BCP at a specific decision level.

- Given a partial conflict graph corresponding to the decision level of the conflict, a unique implication point (UIP) is any node other than the conflict node that is on all paths from the decision node to the conflict node.

First UIP: the UIP closest to the conflict node

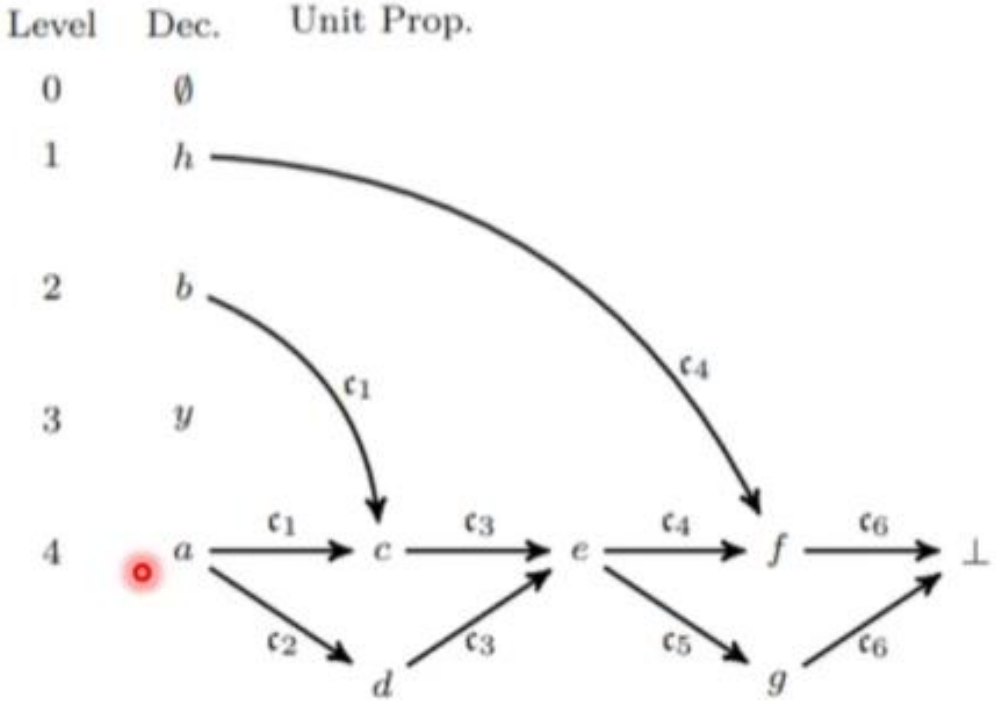


partial implication graph of level 5

Implication Graph

Implication Graph

$$\mathcal{F}_2 = \mathfrak{c}_1 \wedge \mathfrak{c}_2 \wedge \mathfrak{c}_3 \wedge \mathfrak{c}_4 \wedge \mathfrak{c}_5 \wedge \mathfrak{c}_6$$
$$= (\bar{a} \vee \bar{b} \vee c) \wedge (\bar{a} \vee d) \wedge (\bar{c} \vee \bar{d} \vee e) \wedge (\bar{h} \vee \bar{e} \vee f) \wedge (\bar{e} \vee g) \wedge (\bar{f} \vee \bar{g})$$



(a) Implication graph

$x \in \mathbb{V} \cup \{\bot\}$	$\nu(\cdot)$	$\delta(\cdot)$	$\alpha(\cdot)$
h	1	1	$\mathfrak{d}$
b	1	2	$\mathfrak{d}$
y	1	3	$\mathfrak{d}$
a	1	4	$\mathfrak{d}$
c	1	4	$\mathfrak{c}_1$
d	1	4	$\mathfrak{c}_2$
e	1	4	$\mathfrak{c}_3$
f	1	4	$\mathfrak{c}_4$
g	1	4	$\mathfrak{c}_5$
$\bot$	–	–	$\mathfrak{c}_6$

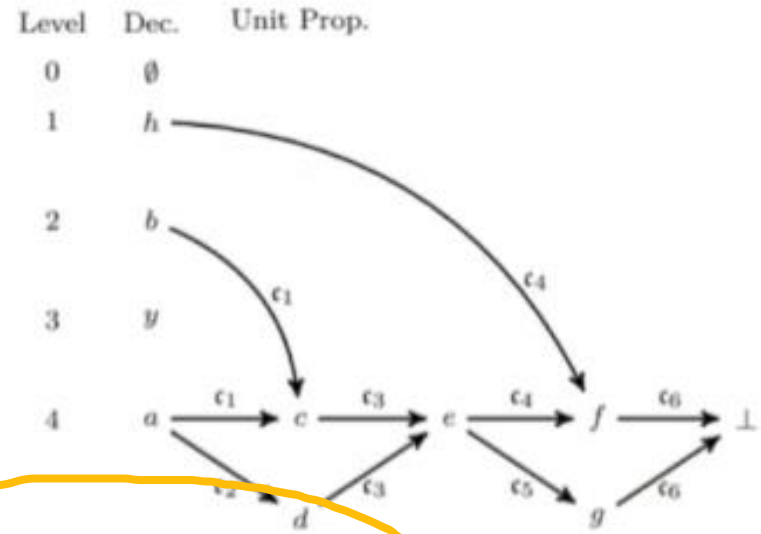
(b) State of variables

For variable x:

$v(x)$: *the value*
$\delta(x)$: *the decision level*
$\alpha(x)$: *the reason clause*

Conflict Analysis

$$\mathcal{F}_2 = \mathfrak{c}_1 \wedge \mathfrak{c}_2 \wedge \mathfrak{c}_3 \wedge \mathfrak{c}_4 \wedge \mathfrak{c}_5 \wedge \mathfrak{c}_6$$
$$= (\bar{a} \vee \bar{b} \vee c) \wedge (\bar{a} \vee d) \wedge (\bar{c} \vee \bar{d} \vee e) \wedge (\bar{h} \vee \bar{e} \vee f) \wedge (\bar{e} \vee g) \wedge (\bar{f} \vee \bar{g})$$



$x \in \mathbb{V} \cup \{\bot\}$	$\nu(\cdot)$	$\delta(\cdot)$	$\alpha(\cdot)$
h	1	1	∂
b	1	2	∂
y	1	3	∂
a	1	4	∂
c	1	4	$\mathfrak{c}_1$
d	1	4	$\mathfrak{c}_2$
e	1	4	$\mathfrak{c}_3$
f	1	4	$\mathfrak{c}_4$
g	1	4	$\mathfrak{c}_5$
$\bot$	–	–	$\mathfrak{c}_6$

从队列（类似于OPEN表）取出

Conflict analysis in an FIFO fashion, starting from the conflict.

Step	Var Queue	Extract Var	Antecedent	Recorded Lits	Added to Queue
0	–	$\bot$	$\mathfrak{c}_6$	$\emptyset$	$\{f, g\}$
1	$[f, g]$	f	$\mathfrak{c}_4$	$\{\bar{h}\}$	$\{e\}$
2	$[g, e]$	g	$\mathfrak{c}_5$	$\{\bar{h}\}$	$\emptyset$
3	$[e]$	e	$\mathfrak{c}_3$	$\{\bar{h}, \bar{e}\}$	$\emptyset$
6	$[]$	–	–	$\{\bar{h}, \bar{e}\}$	–

In each step, for the reason clause,

- all literals at levels smaller than the current one are added to the clause being learned
- literals at current level are added to the queue for analyses.

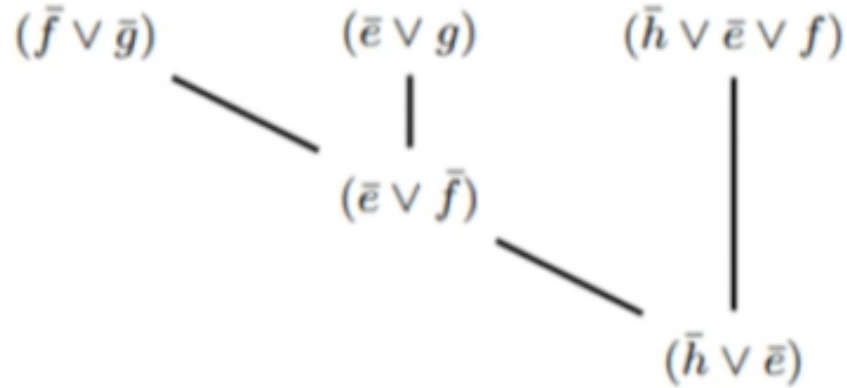
低于当前层，取反，然后直接加入学习子句（recorded lists）

同层的变量直接加入队列

Conflict Analysis and Resolution

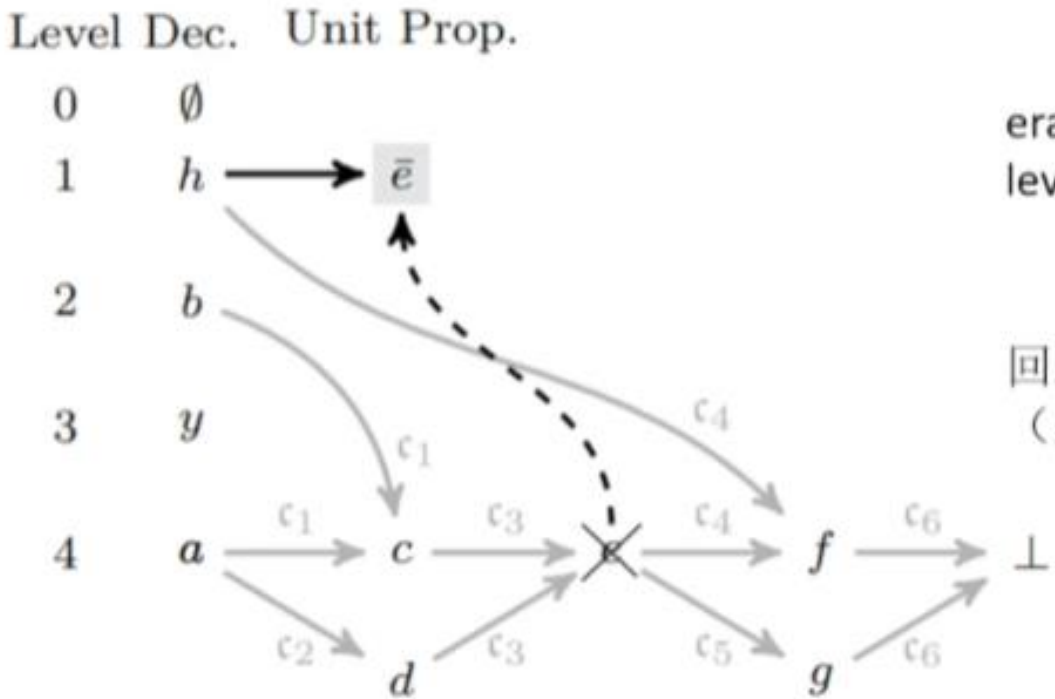
$$\mathcal{F}_2 = \mathfrak{c}_1 \wedge \mathfrak{c}_2 \wedge \mathfrak{c}_3 \wedge \mathfrak{c}_4 \wedge \mathfrak{c}_5 \wedge \mathfrak{c}_6$$
$$= (\bar{a} \vee \bar{b} \vee c) \wedge (\bar{a} \vee d) \wedge (\bar{c} \vee \bar{d} \vee e) \wedge (\bar{h} \vee \bar{e} \vee f) \wedge (\bar{e} \vee g) \wedge (\bar{f} \vee \bar{g})$$

Step	Var Queue	Extract Var	Antecedent	Recorded Lits	Added to Queue
0	–	$\perp$	$\mathfrak{c}_6$	$\emptyset$	$\{f, g\}$
1	$[f, g]$	f	$\mathfrak{c}_4$	$\{\bar{h}\}$	$\{e\}$
2	$[g, e]$	g	$\mathfrak{c}_5$	$\{\bar{h}\}$	$\emptyset$
3	$[e]$	e	$\mathfrak{c}_3$	$\{\bar{h}, \bar{e}\}$	$\emptyset$
6	[]	–	–	$\{\bar{h}, \bar{e}\}$	–



Conflict Driven Backtracking

$$\mathcal{F}_2 = c_1 \wedge c_2 \wedge c_3 \wedge c_4 \wedge c_5 \wedge c_6$$
$$= (\bar{a} \vee \bar{b} \vee c) \wedge (\bar{a} \vee d) \wedge (\bar{c} \vee \bar{d} \vee e) \wedge (\bar{h} \vee \bar{e} \vee f) \wedge (\bar{e} \vee g) \wedge (\bar{f} \vee \bar{g}) \wedge (\bar{h} \vee \bar{e})$$



backtrack with learnt clause $\bar{h} \vee \bar{e}$

erase all decisions and implications made after that level.

回退到学习子句的第二深
（就是除了当前层之外，最深的一层）

Conflict Driven Backtracking

Two special cases:

一元学习子句：子句只有一个变量，意味着这个literal必须成立
操作：直接回退到level 0，就是没有任何赋值的情况

- when learning a unary clause, the solver backtracks to the ground level;

- when the conflict is at the ground level, the backtracking level is set to −1 and the solver exits and declares the formula to be unsatisfiable.

如果某个子句被 falsified或推出 ⊥level 0：意味着某个子句被 falsified或推出 ⊥不做任何假设，就已经矛盾
直接exit

Learnt Clauses and UNSAT Certificate

不可满足性证明

- The input is a CNF formula F and a queue Q of lemmas reprepresenting a refutation of F.
- Lemmas are sorted in chronological order as learned by the SAT solver.

Q就是右边的学习子句集，每次取出其中的一个

RUPchecker (CNF formula F, queue Q of lemmas)

```
1  while Q is not empty
2      L := Q.pop()
3      F' := BCP(F ∪ ¬L)
4      if ∅ ∉ F' then return "checking failed"
5      F := BCP(F ∪ L)
6      if ∅ ∈ F then return "unsatisfiable"
7  return "all lemmas validated"
```

代入第一个学习子句的非，代入CNF中，1，2两个子句发生冲突（分别化简成了+-x4）

中间的两个return：
1.检查这个 lemma（L）是不是合法的
2.检查整个公式是不是已经 UNSAT

Certificate formate: RUP, RAT, DRAT, DRAT-trim

RUP: Reverse Unit Propagation

CNF formula

```
p cnf 4 16
 1  2  3  4 0
 1  2  3 -4 0
 1  2 -3  4 0
 1  2 -3 -4 0
 1 -2  3  4 0
 1 -2  3 -4 0
 1 -2 -3  4 0
 1 -2 -3 -4 0
-1  2  3  4 0
-1  2  3 -4 0
-1  2 -3  4 0
-1  2 -3 -4 0
-1 -2  3  4 0
-1 -2  3 -4 0
-1 -2 -3  4 0
-1 -2 -3 -4 0
```

smallest RUP proof

```
1 2 3 0
1 2 0
1 3 0
1 0
2 3 0
2 0
3 0
0
```

表示x123的析取，0是终止符号

左边这个cnf集合相当于是对x1-4每个变量都进行一个正负遍历所得到的子句

CDCL Algorithm – CDCL

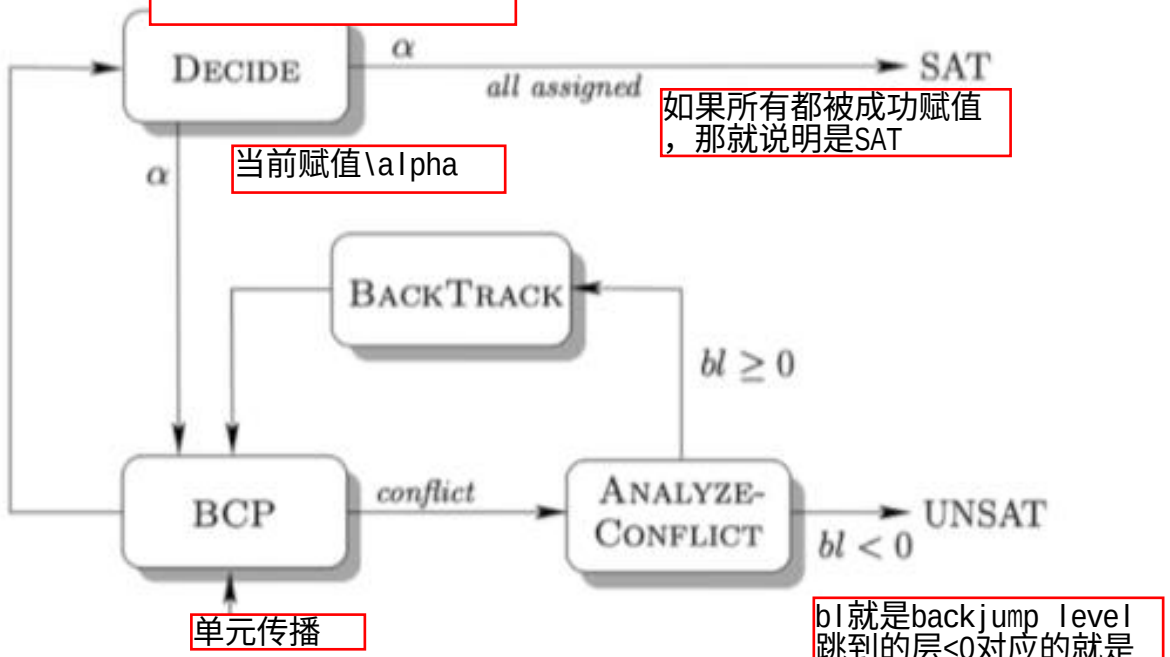
冲突分析

Analyze-Conflict : non-chronological backtracking + clause learning + vivification

决策

Decide : Branching strategy and phasing strategy

做一个猜测赋值【决策】



如果所有都被成功赋值，那就说明是SAT

当前赋值\alpha

单元传播

bl就是backjump level跳到的层<0对应的就是严重冲突

- Clause learning
- Clause management
- Lazy data structures
- Restarting
- Branching
- Phasing
- Mode Switching
- ...

CDCL Heuristics – Branching heuristics

- Static branching heuristic: e.g. Ordered BDDs

这几页是集中讲“如何选择下一个变量”的

- Dynamic literal individual sum heuristic (DLIS) [Copty et al. 2001]
  - counts the number of clauses that include this literal and are not yet satisfied,
  - the literal with the largest count is chosen.

其中最有代表性的就是
VSIDS：前面讲的，给变量打分
VMTF：更简单，直接将最近发生冲突的变量移到最前面

- Variable state independent decaying sum(VSIDS) and its variants

- Variable move to front (VMTF) [Ryan2004, BiereFröhlich, 2015]
  - mark the last learned literals as the most important – they are moved to the front of the queue and will be selected next (last in first out)

CDCL Heuristics – Branching heuristics

- Learning-Rate Branching (LRB)[LiangGaneshPoupartCzarnecki, 2016]
  - picks a variable to maximize a metric called learning rate
  - The learning rate of a variable x at interval I is $r_x = \frac{P(x,I)}{|I|}$ 引入学习率来来进行更新
    - I is the sequence of conflicts that occurred between the assignment of x until it transitioned back to unassigned,
    - P(x,I) measures the number of conflicts in I, for which x occurred in at least one clause in clause learning
  - Futher: Compute score A_v for each variable x, select variable with highest score.

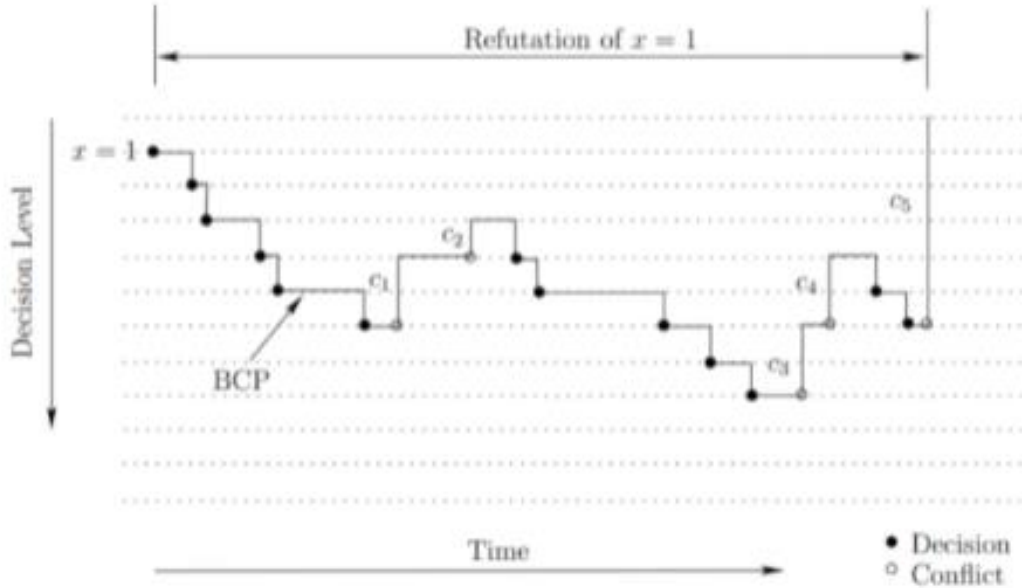
$$A_x = (1-\alpha)A'_x + \alpha r_x$$

- Dynamic switching between multiple heuristics

CDCL Heuristics – Phasing heuristics

Modern phasing heuristic: Phase saving + Rephasing

- Phase : the value which variable should assign when branching.



This drawing illustrates the work done by the SAT solver to refute this wrong decision---x=1.

- Only some of the work was necessary for creating c5, refuting this decision, and computing the backtracking level.
- The "wasted work" (which might, after all, become useful later on) is due to the imperfection of the decision heuristic.

Phase saving: keep the last assignment, and reusing it whenever the variable's value has to be decided again

CDCL Heuristics – Phasing heuristics

- Phase saving can considered an intensification strategy
- Complement it with a diversification strategy should be beneficial
- The idea of rephasing is to regularly reset saved phases. [Biere,2010]
- In principle, saved phases can be set arbitrarily, as phase selection does not influence correctness nor termination of CDCL
  - Random
  - Default 0, 1
  - Flip the saved phase
  - Local search
  - ...

CDCL Heuristics – Learnt Clause Removal

- Reason why **Clause Database Reduction**
  - Not all of them are helpful;
  - UP gets slower with memory consumption.

需要适当删除一些学习子句，
因为不是所有都有用
而且随着记忆冗余可能效率会
下降

CDCL Heuristics – Learnt Clause Removal

- Reason why **Clause Database Reduction**
  - Not all of them are helpful;
  - UP gets slower with memory consumption.

需要适当删除一些学习子句，
因为不是所有都有用
而且随着记忆冗余可能效率会下降

- Measurement criteria of clauses
  - least recently used (LRU) heuristics: discard clauses not involved in recent conflict clause generation
  - **Literal Block Distance(LBD): number of distinct decision levels in learnt clauses**, proposed in glucose. [AudemardSimon,2009]

LBD=在这个 learned clause 中，文字来自的“不同 decision level”的个数

CDCL Heuristics – Learnt Clause Removal

3-tired clause management [Chanseok Oh, 15SAT]

- *core* are clauses with LBD≤ 3;
- *mid_tire* retain recently used clause with LBD up to 6;
- *local* saving other clauses.

通过LBD的值和使用频率来给子句分成三层

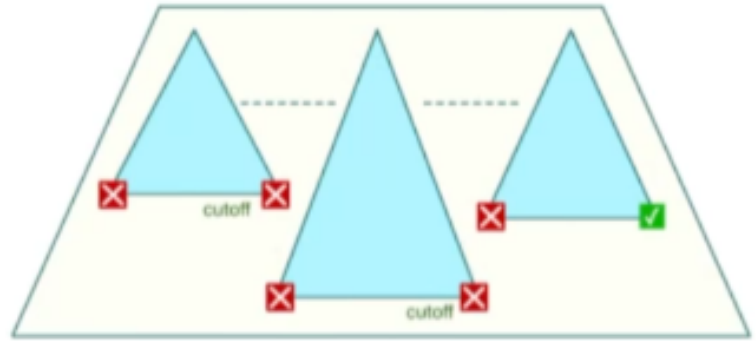
3-tier method

- *core* never be removed;
- Periodically remove half *local* clauses based on score.
- Periodically move some recently not used clauses in *mid_tire* to *local*.
- move clauses encounter in *local* many times to *mid_tire*, and same from *mid_tire* to *core*.

CDCL Heuristics – Effective Restart

Restart: periodical traceback to 0 decision level.

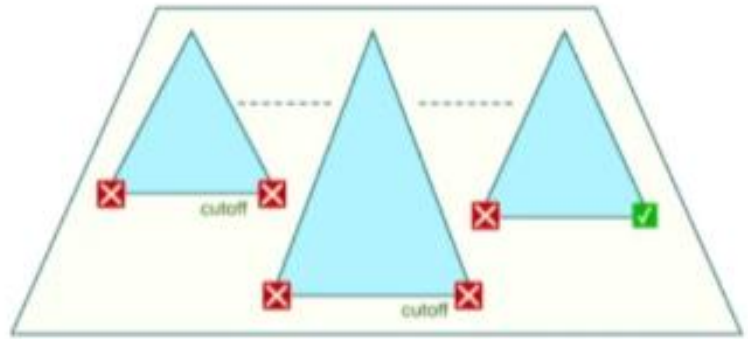
- clause learning and search restarts correspond to a proof system as powerful as general resolution, and stronger than DPLL proof system; practically effective.



CDCL Heuristics – Effective Restart

Restart: periodical traceback to 0 decision level.

- clause learning and search restarts correspond to a proof system as powerful as general resolution, and stronger than DPLL proof system; practically effective.



Restart policies:

- Luby series: 1 1 2 1 1 2 4 1 1 2 1 1 2 4 8 ... [LubySinclairZuckerman, 1993]
  - priori optimal strategy

- Glucose restart (rapid) : When average LBD of some current learnt clauses is great than the average LBD of all learnt clauses. [AudemardSimon,2012]

CDCL Heuristics – Effective Restart

An issue : in order to find a model, the solver must generate long assignments.

- Luby-style restart: the (non-monotonically) increasing intervals between successive restarts
- Glucose-style restart: restart can be blocked and delayed whenever the current assignment looks promising
  - for example, if the trail length has increased by a predefined factor since the latest restart [AudemardSimon, 2012]

- A conjecture: rapid restart generally helps deriving a refutation proof, while remaining in the current branch increases the chance of reaching a model
  - interleave "stabilizing" mode (no restarts) and "focused" mode [Chanseok Oh,15SAT]

Summary

Information of this part:

- History of SAT solving
- key data structure and implementation of CDCL
  - Watching literals
  - clause learning
- Understand the core ideas of CDCL (know why)
- Know important engineering issues